

Introdução

Python para ML - Parte 3

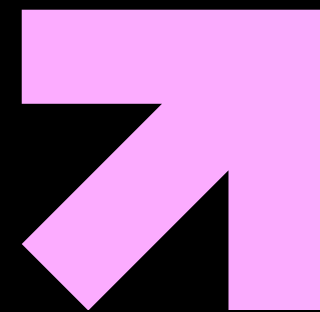
Manipulação de arquivos e List Comprehensions

Roteiro da Semana

OBJETIVO DA SEMANA:
CONSOLIDAR OS
FUNDAMENTOS DE PYTHON
ANTES DE PARTIMOS PARA
MACHINE LEARNING.

Tópico	O que você vai aprender
Leitura de arquivos	Abrir e ler arquivos de texto (.txt) e CSV
Escrita de arquivos	Salvar dados em arquivos
List Comprehensions	Criar listas de forma mais eficiente e elegante
Exercícios práticos	Aplicar os conceitos em problemas reais

POR QUE APRENDER MANIPULAÇÃO DE ARQUIVO



Exemplo real: Um dataset de vendas vem em um arquivo venda.csv. Você precisa ler esse arquivo para analisar os dados.

Contexto para Machine Learning:

- Os dados que usamos em ML geralmente estão em arquivos (CVS, Excel, JSON, etc.)
- Precisamos saber ler esses arquivos para dentro do Python
- Também precisamos saber salvar resultados e dados processados



ABRINDO ARQUIVOS COM OPEN()

```
arquivo = open("nome_do_arquivo.txt", "r")
conteudo = arquivo.read()
print(conteudo)
arquivo.close()
```

Importante: Sempre feche o arquivo com `.close()`!

Modos de abertura:

Modo	Significado
"r"	Leitura (read) - padrão
"w"	Escrita (write) - sobrescreve o arquivo
"a"	Adicionar (append) - acrescenta ao final
"x"	Criar (create) - só se não existir

MELHOR PRÁTICA: USANDO WITH

Por que usar with?

- Não precisa lembrar de fechar o arquivo (fecha automaticamente)
- Código mais limpo e seguro

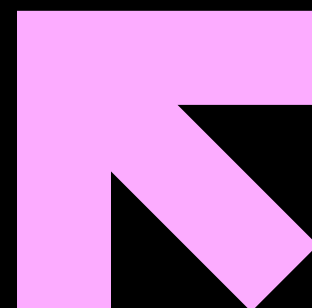
Exemplo:

```
with open("dados.txt", "r") as arquivo:  
    conteudo = arquivo.read()  
    print(conteudo)  
  
# O arquivo é fechado automaticamente aqui
```

Execute você mesmo:

1. Crie um arquivo teste.txt no mesmo diretório do seu notebook
2. Escreva algo dentro dele
3. Execute o código acima

LENDO ARQUIVOS LINHA POR LINHA

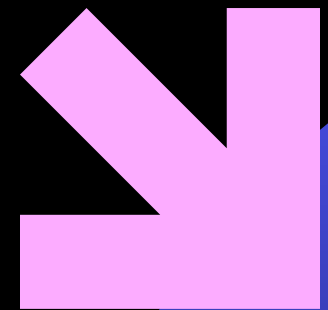


Quando usar: Arquivos muito grandes
que não cabem na memória

```
with open("dados.txt", "r") as arquivo:
    for linha in arquivo:
        print(linha.strip()) # .strip() remove quebras de linha
```

Outro métodos úteis:

Método	O que faz
arquivo.read()	Lê o arquivo inteiro como uma string
arquivo.readline()	Lê uma única linha
arquivo.readlines()	Lê todas as linhas e retorna uma lista



Escrevendo em arquivos

Modo escrita (“w”) - sobrescreve o arquivo:

```
with open("saida.txt", "w") as arquivo:  
    arquivo.write("Primeira linha\n")  
    arquivo.write("Segunda linha\n")
```

Modo append (“a”) - adiciona ao final:

```
with open("saida.txt", "a") as arquivo:  
    arquivo.write("Terceira linha (adicionada)\n")
```

Importante:

- “w” - apaga tudo que existia e começa do zero
- “a” - mantém o conteúdo existente e adiciona no final

Problema: Calcular os quadrados de 1 a 10 e salvar em um arquivo.

```
with open("quadrados.txt", "w") as arquivo:
    for i in range(1, 11):
        quadrado = i ** 2
        arquivo.write(f"{i} ao quadrado é {quadrado}\n")

print("Arquivo 'quadrados.txt' criado com sucesso!")
```

Resultado no arquivo:

```
1 ao quadrado é 1
2 ao quadrado é 4
3 ao quadrado é 9
...
10 ao quadrado é 100
```

EXEMPLO PRÁTICO: SALVANDO RESULTADOS

LENDO ARQUIVOS CSV (PRÉVIA)

O que é CSV?

CSV = Comma-Separated Values (valores separados por vírgula)

É o formato mais comum para datasets de Machine Learning.

Exemplo de arquivo CSV (dados.csv):

```
nome,idade,cidade
```

```
Ana,25,São Paulo
```

```
João,30,Rio de Janeiro
```

```
Maria,22,Belo Horizonte
```

Lendo um CSV manualmente:

```
with open("dados.csv", "r") as arquivo:
    linhas = arquivo.readlines()

cabecalho = linhas[0].strip().split(",")
print("Cabeçalho:", cabecalho)

for linha in linhas[1:]:
    valores = linha.strip().split(",")
    print(f"Nome: {valores[0]}, Idade: {valores[1]}, Cidade: {valores[2]}")
```

Nota: Na semana 5 vamos aprender o Pandas, que facilita muito a leitura de CSVs!

Introdução a list Comprehensions

O que é?

Uma forma concisa e eficiente de criar listas em Python.

Problema: Criar uma lista com os quadrados de 0 a 9.

Jeito tradicional (sem list comprehension):

```
quadrados = []  
for i in range(10):  
    quadrados.append(i ** 2)  
print(quadrados) # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

Com list comprehension:

```
quadrados = [i ** 2 for i in range(10)]  
print(quadrados) # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

Sintaxe: [expressão for item in iterável]

LIST COMPREHENSION COM CONDIÇÃO

Adicionamos um if para filtrar:

```
# Números pares de 0 a 20
pares = [i for i in range(21) if i % 2 == 0]
print(pares) # [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

Sintaxe com condicional:

[valor_se_verdadeiro if condição
valor_se_falso for item in iterável]

Com if-else (condicional ternário):

```
# Par ou ímpar para números de 0 a 9
resultado = ["par" if i % 2 == 0 else "ímpar" for i in range(10)]
print(resultado) # ['par', 'ímpar', 'par', 'ímpar', 'par', 'ímpar', 'par', 'ímpar', 'pa  
r', 'ímpar']
```

EXEMPLOS PRÁTICOS DE LIST COMPREHENSION

Problema	Solução com List Comprehension
Dobro dos números de 1 a 10	<code>[i * 2 for i in range(1, 11)]</code>
Letras maiúsculas de uma palavra	<code>[letra.upper() for letra in "python"]</code>
Números divisíveis por 3 até 30	<code>[i for i in range(1, 31) if i % 3 == 0]</code>
Converter lista de strings para inteiros	<code>[int(x) for x in ["1", "2", "3"]]</code>

LIST COMPREHENSION VS LOOP TRADICIONAL

Comparaçã de performance:

```
# Jeito tradicional (mais lento)
quadrados_trad = []
for i in range(1000000):
    quadrados_trad.append(i ** 2)

# Com list comprehension (mais rápido)
quadrados_lc = [i ** 2 for i in range(1000000)]
```

Vantagens da List Comprehension:

- Código mais curto e legível
- Geralmente mais rápido que loops tradicionais
- Fácil de combinar com filtros (if)

Desvantagem:

- Pode ficar difícil de ler se for muito complexo (nesse caso, prefira o loop tradicional)

Exemplo integrado: lendo arquivo e processando

Problema: Ler um arquivo com números (um por linha), calcular o dobro e salvar em outro arquivo.

Arquivo de entrada (numeros.txt): 1, 2, 3, 4, 5

Resultado em dobros.txt: 2, 4, 6, 8, 10

Solução:

```
# Ler os números do arquivo
with open("numeros.txt", "r") as entrada:
    numeros = [int(linha.strip()) for linha in entrada]

# Calcular o dobro usando list comprehension
dobros = [n * 2 for n in numeros]

# Salvar em um novo arquivo
with open("dobros.txt", "w") as saida:
    for dobro in dobros:
        saida.write(f"{dobro}\n")

print("Processamento concluído!")
```

EXERCÍCIOS PARA FAZER EM CASA

Instruções: Crie um notebook no Google Colab, resolva os exercícios abaixo e suba na pasta semana4/ do repositório.

Exercício 1 – Leitura de arquivo

Crie um arquivo chamado frutas.txt com 5 nomes de frutas (um por linha). Escreva um programa que leia o arquivo e exiba cada fruta em maiúsculas.

Exercício 2 – Escrita de arquivo

Crie um programa que gere os números de 1 a 50, calcule o cubo de cada um, e salve no formato "O cubo de X é Y" em um arquivo chamado cubos.txt.

Exercício 3 – Integrado (arquivo + list comprehension)

Crie um arquivo idades.txt com 10 idades (uma por linha). Leia o arquivo, use list comprehension para filtrar apenas as idades maiores que 18, e salve o resultado em um arquivo maiores.txt.

Obrigada!